Foodies Web Application: Connected Analysis

Testing, requirements, architecture, journey, use cases, and consistency review

> Scope note: The case describes a product called Foodies. The task also uses the name Swiggy in the application-overview instruction. This report therefore treats Foodies as the application being analysed and uses only the behaviours explicitly supplied in the case. No unsupported claims are made about Swiggy's or Foodies' internal technology.

Executive summary

Foodies is analysed as a web application because customers and operational users interact with the system through browser-based screens that send actions to shared application services and receive updated results. The three testing failures expose three different control points: (1) stale cart/menu information, (2) a broken connection between successful payment and order confirmation, and (3) missing authorization when an order identifier is changed. A consistent design must validate current prices, reconcile payment with order creation/confirmation, and authorize every order lookup against the requesting customer's identity.

1. Application overview

Why it is a web application. A web application accepts user actions through an interactive client interface and processes those actions using application-side logic, with results returned to the user. The case gives two direct behavioural indicators:

Behaviour 1 — browse, cart, payment, tracking: customers browse nearby restaurants, add items to a cart, pay, and track accepted orders. This is a sequence of interactive user actions and visible system responses rather than a static information page.

Behaviour 2 — multiple operational roles: restaurant partners accept/reject orders and update preparation status, delivery partners update delivery progress, and support agents investigate failed or disputed orders. The same application therefore supports different users and role-specific workflows.

These behaviours are sufficient to establish the application character for this analysis without assuming a particular programming language, framework, hosting platform, or database product.

2. Actors and requirements

Code	Actor	Goal
A1	Customer	Find food, create/pay for an order, and track its accepted delivery.
A2	Restaurant Partner	Receive orders, accept/reject them, and update preparation status.
A3	Delivery Partner	View assigned delivery jobs and update delivery progress.
A4	Support Agent	Investigate failed or disputed orders and determine appropriate support action.
A5	Payment Gateway	Return a payment outcome for a payment attempt.
A6	Foodies Application	Coordinate the shared workflow and maintain order state (system actor/component boundary, not a human actor).

Numbered functional requirements

#	Actor	Functional requirement
FR1	Customer	Browse nearby restaurants and menus so the customer can choose food.

#	Actor	Functional requirement
FR2	Customer	Create and maintain a cart from selected menu items before payment.
FR3	Customer	Place an order and receive an order confirmation after the payment/order workflow succeeds.
FR4	Customer	View the status of the customer's own accepted order and track delivery progress.
FR5	Restaurant Partner	Accept or reject a received order.
FR6	Restaurant Partner	Update preparation status for an accepted order.
FR7	Delivery Partner	View assigned delivery jobs and update delivery progress.
FR8	Support Agent	Investigate failed or disputed orders and record/support an appropriate resolution.
FR9	Payment Gateway	Return a payment outcome that the application can use to decide the payment state.

3. Components and responsibilities

Component	Responsibility in this analysis
Frontend	Presents screens for browsing, cart, checkout, order status, restaurant actions, delivery actions, and support work. Captures user actions and displays responses/status.
Backend	Applies business decisions and workflow rules: validates cart/order data, decides whether an order may proceed, coordinates payment outcomes, applies role/ownership checks, and changes order state.
Database	Persists business state such as menu/cart/order information and status so later requests can use consistent stored information.
Payment Gateway	External payment participant that receives a payment request and returns a payment outcome. Foodies must interpret that outcome as part of the order workflow.

Implementation boundary: The responsibilities above are architectural roles inferred from the behaviours and requirements in the case. They do not assert a specific framework, cloud service, database engine, internal service decomposition, or API design. Where this report discusses how a component would enforce a rule, that is an inference for analysis, not a claim about the real implementation of Swiggy.

4. Place Order journey

Stage	Action	Frontend / UI	Backend + persistent state	Visible outcome
1. Customer action	Customer selects restaurant/menu items and adds an item to the cart.	Frontend records the action and shows the cart.	Cart contents are persisted or otherwise retained as current order-input state.	Updated cart is displayed.
2. Price validation	Customer proceeds to checkout.	Backend rechecks current item/price information before accepting the order.	The authoritative order price is stored from the validated data.	Checkout either continues with the current price or asks the customer to review a changed price.
3. Payment	Customer confirms payment.	Backend sends the payment request to the Payment Gateway and receives the outcome.	Payment/order state is updated according to the returned outcome.	The customer sees payment progress/result.
4. Order creation/confirmation	Payment succeeds.	Backend decides whether the order can move to confirmed/accepted workflow state and links the successful payment to that order.	Order state becomes a confirmed/placed state only when the required conditions are satisfied.	Customer receives order confirmation and an order identifier.

Stage	Action	Frontend / UI	Backend + persistent state	Visible outcome
5. Restaurant processing	Restaurant partner receives the order.	Backend makes the order available to the restaurant partner and accepts the partner's accept/reject decision.	Order state changes to accepted/rejected; preparation status is retained for accepted orders.	Customer sees the resulting order status.
6. Delivery	Accepted order is assigned to a delivery partner.	Backend makes the assigned job available and accepts delivery-progress updates.	Delivery progress is persisted.	Customer sees updated delivery status.

5. Problem analysis

P1 — Menu price changes after add-to-cart

Who detects: Customer or tester detects the mismatch when the cart/checkout price differs from the current menu price.

Who decides: The backend should decide the authoritative price at checkout by revalidating the current menu item/price before creating the payable order. This is an inferred control, not a claim about the existing implementation.

State change: Cart/checkout state is corrected or held for customer review; the final order price must reflect the validated value.

What the user sees: The customer should see a clear price-change message and the updated total before confirming payment, rather than silently paying the stale price.

P2 — Payment succeeds but no order confirmation

Who detects: Customer detects it because payment has succeeded but the expected confirmation/order status is missing.

Who decides: The Foodies backend must reconcile the successful Payment Gateway outcome with order creation/confirmation and determine whether the order is safely confirmable.

State change: Payment state becomes successful; independently, the order must reach a consistent confirmed/placed state. If reconciliation fails, the system should retain enough state for support/recovery rather than presenting an ambiguous result.

What the user sees: Customer should see either a confirmed order with its identifier or a clear processing/failure message. A silent success with no order visibility is inconsistent.

P3 — Customer changes order identifier to view another customer's order

Who detects: The security/control test detects unauthorized access when an identifier is manipulated.

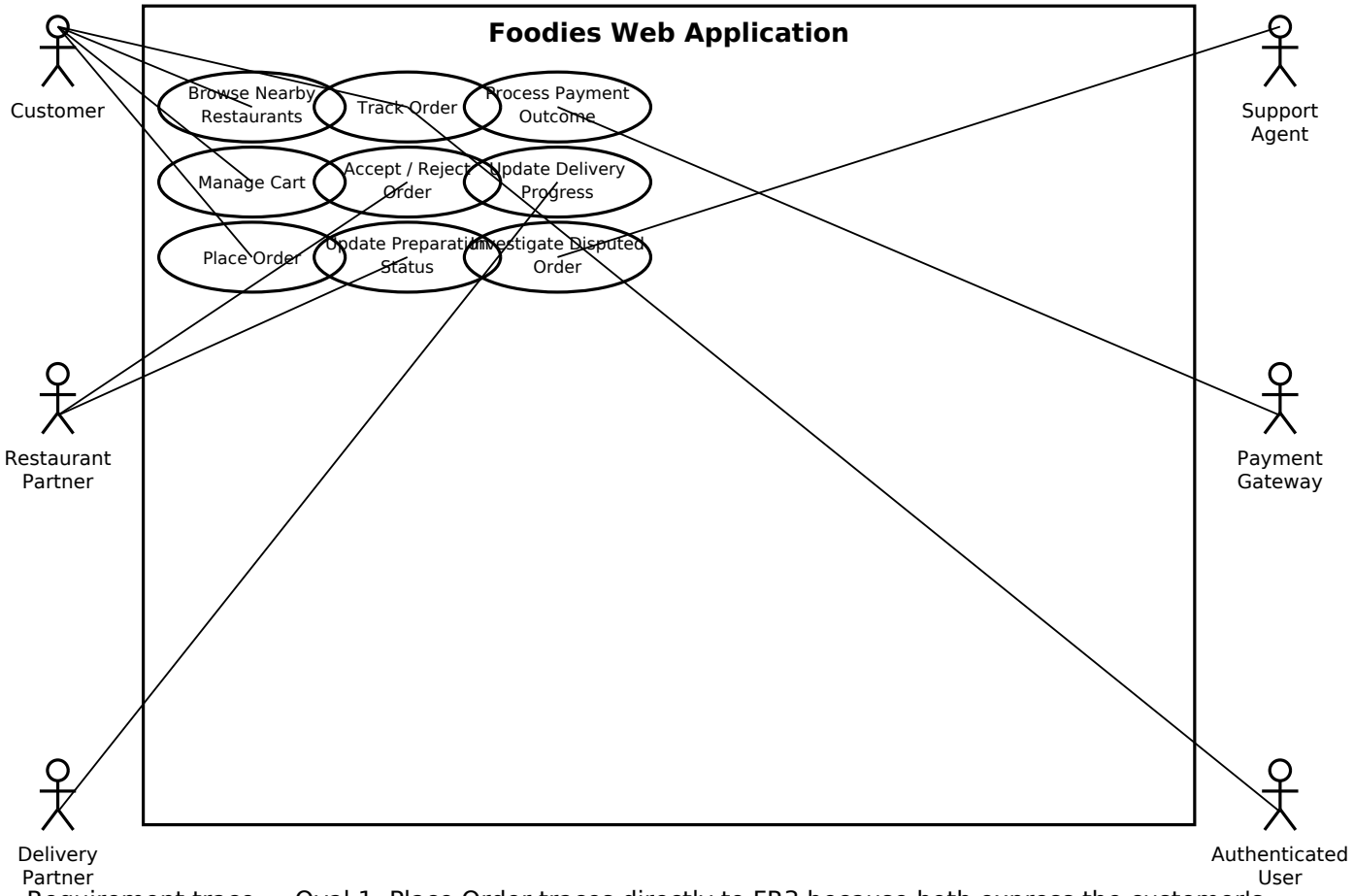
Who decides: The backend must decide authorization using the authenticated customer's ownership/permission, not merely whether the supplied identifier is syntactically valid.

State change: No protected order data should be exposed or changed for an unauthorized customer; the access attempt should be rejected.

What the user sees: The customer sees an access-denied/not-found style result rather than another customer's order details. Support/admin access may follow separate authorized rules.

6. Use case diagram

The diagram uses one named system boundary, six external actors, and nine verb-object use cases. Associations connect each actor only to use cases that match the actor's stated goal.



Requirement trace — Oval 1: Place Order traces directly to FR3 because both express the customer's goal of creating an order and receiving confirmation after the payment/order workflow.

Requirement trace — Oval 2: Update Preparation Status traces directly to FR6 because both express the restaurant partner's goal of changing preparation status for an accepted order.

Association note: The Payment Gateway is associated with Process Payment Outcome, not with the customer's cart or delivery activities. The Authenticated User is shown as a security context for Track Order; the critical authorization rule is that the application must check ownership/permission before exposing an order.

7. Consistency review

Contradiction/gap found: The scenario says a payment can succeed while the customer receives no order confirmation. A simplistic journey could therefore treat successful payment as equivalent to confirmed order, which would make the failure impossible to represent clearly.

Correction: The report separates payment state from order state. Payment success is an input to the backend decision; it does not by itself prove that an order has reached confirmed/placed state. The corrected journey therefore includes an explicit reconciliation/confirmation step, and the problem analysis requires the user to see either a confirmed order or a clear processing/failure outcome.

Security consistency check: The same separation is applied to order viewing: possession of an order identifier is not treated as permission to view the order. Authorization is a backend decision based on the requesting user's ownership/role.

Final consistency position: Menu data, payment state, order state, and authorization are distinct concerns. Keeping those decisions explicit makes the requirements, journey, use cases, and testing

analysis agree with one another.

Conclusion

The pilot should not be approved until the three observed behaviours have clear controls and visible outcomes: current-price validation before final payment, reliable reconciliation between successful payment and order confirmation, and server-side authorization for every order access. These controls address the failures while remaining implementation-neutral.

Prepared as a requirements and system-analysis report based solely on the supplied case description.